Reflection

Trent Gwathney | IS601 | Module 10

Adding users to the calculator helped me understand how database models and API schemas serve different purposes. SQLAlchemy defines stored fields and unique constraints, while Pydantic checks incoming requests. A separate response schema keeps password hashes out of the API output (not something I want showing up in a response).

Argon2id produces a salted hash instead of storing the original password. Testing correct and incorrect passwords helped me understand verification, and seeing different hashes for identical passwords made the role of salts clearer.

One challenge was extending the project without disrupting my previous database work. An Alembic migration created the user table in a separate schema, preserving the earlier tables. Transactions and savepoints let PostgreSQL tests exercise real constraints and roll back their changes. Windows had also reserved port 8000, so I switched to 18000 (apparently Windows had other plans for that port).

The image scan showed that passing application tests did not mean the container was ready to publish (so passing tests was not quite the finish line). Switching to an updated Alpine base resolved the reported vulnerabilities. GitHub Actions automated testing, scanning, publishing, and checking the pulled image.

I feel more confident with validation, password hashing, and database tests. I still want more practice with migrations, interpreting vulnerability reports, and adding authentication and authorization.